Package 'GTAPViz'

March 31, 2025

Title R Package for Automating 'GTAP' Data Processing and Visualization

Version 1.0.0

Author Pattawee Puangchit <ppuangch@purdue.edu>

Description An R package designed to streamline the processing and visualization of 'GTAP' model results. While primarily built for 'GTAP' simulations, it also supports other .har and .sl4 datasets. As an extension of HARplus, GTAPViz simplifies 'GTAP' data extraction and visualization, reducing manual effort and enhancing efficiency. With intuitive commands and automated adjustments, users can create, customize, and export high-quality visualizations for economic analysis and research reporting with minimal coding.

License MIT + file LICENSE

Encoding UTF-8

RoxygenNote 7.3.2

BugReports https://github.com/bodysbobb/GTAPViz/issues

URL https://bodysbobb.github.io/GTAPViz/

Imports HARplus,
ggplot2,
dplyr,
tidyr,
openxlsx,
colorspace,
grDevices,
scales,
utils,
methods,
stringdist,
stats,
glue,
openxlsx2

VignetteBuilder knitr

Suggests knitr,
rmarkdown,
devtools

Depends R (>= 3.5)

Contents

`add_mapping_info` *Add Mapping Information to GTAP Data*

Description

Adds descriptions and unit information to GTAP data based on a specified mapping mode. Supports external mappings or default GTAPv7 mappings, allowing users to enrich datasets with standardized metadata.

Usage

```
add_mapping_info(
  data_list,
  external_map = NULL,
  mapping = "GTAPv7",
  description_info = TRUE,
  unit_info = TRUE
)
```

Arguments

`data_list`	A data structure containing GTAP variables.
`external_map`	Optional data frame with mapping information (must include "Variable", "Description", and "Unit" columns).
`mapping`	Character. Mapping mode: - ‘"GTAPv7"‘ (default): Uses standard GTAPv7 definitions. - ‘"Yes"‘: Uses only the supplied descriptions and units from ‘external_map‘. - ‘"No"‘: Does not add any descriptions or units. - ‘"Mix"‘: Prioritizes ‘external_map‘, but falls back to GTAPv7 for missing values.
`description_info`	Logical. If ‘TRUE‘, adds description information to the data.
`unit_info`	Logical. If ‘TRUE‘, adds unit information to the data.

Value

A data structure with added mapping information, preserving the original structure.

Author(s)

Pattawee Puangchit

See Also

convert_units, rename_value

Examples

```
# Load Sample Data:
sl4_data1 <- HARplus::load_sl4x(system.file("extdata/in", "EXP1.sl4",
                                package = "GTAPViz"))

# Get Data by Variable Name
sl4_data1 <- HARplus::get_data_by_var(c("qgdp", "EV"), sl4_data1)

# Add mapping using GTAPv7 defaults
gtap_data <- add_mapping_info(sl4_data1, mapping = "GTAPv7")

# Use an external mapping file
my_mapping <- data.frame(Variable = c("qgdp", "EV"),
                         Description = c("Real GDP", "Welfare"),
                         Unit = c("percent", "millionUSD"))  # <- Fixed closing quote

gtap_data <- add_mapping_info(sl4_data1, external_map = my_mapping,
                              mapping = "Mix")
```

---

auto_gtap_data *Process GTAP Data Automation with Flexible Output Options*

Description

Processes GTAP data from SL4 and HAR files with options for exporting and preparing plot-ready data.

Usage

```
auto_gtap_data(
  experiment,
  project_path = NULL,
  input_path = NULL,
  output_path = NULL,
  sl4_suffix = "",
  har_suffix = "-WEL",
  mapping_info = "GTAPv7",
  process_sl4_vars = NULL,
  process_har_vars = NULL,
```

```
  sl4_mapping_info = NULL,
  har_mapping_info = NULL,
  sl4_extract_method = "group_data_by_dims",
  har_extract_method = "get_data_by_var",
  sl4_priority = NULL,
  har_priority = NULL,
  region_select = NULL,
  sector_select = NULL,
  subtotal_level = FALSE,
  plot_data = FALSE,
  output_formats = NULL,
  sl4_output_name = "sl4.plot.data",
  har_output_name = "har.plot.data",
  macro_output_name = "GTAPMacro",
  add_scenario_ranking = FALSE,
  rank_column = "ScenarioRank"
)
```

Arguments

`experiment`	Character vector. Case names to process.
`project_path`	Character. Path to the project folder with "in" and "out" subfolders.
`input_path`	Character. Path to the input folder. Overrides ‘project_path/in‘ if specified.
`output_path`	Character. Path to the output folder. Overrides ‘project_path/out‘ if specified.
`sl4_suffix`	Character. Custom suffix for SL4 files (e.g., "" or "-custom").
`har_suffix`	Character. Custom suffix for HAR files (e.g., "-WEL").
`mapping_info`	Character. Mapping mode: "GTAPv7" (default), "Yes", "No", or "Mix".
`process_sl4_vars`	Data frame, NULL, or FALSE. Variables to extract from SL4 files. - Set to NULL to extract all variables. - Set to FALSE to skip SL4 processing.
`process_har_vars`	Data frame, NULL, or FALSE. Variables to extract from HAR files. - Set to NULL to extract all variables. - Set to FALSE to skip HAR processing.
`sl4_mapping_info`	Data frame or NULL. Mapping information for SL4 variables (with "Variable", "Description", and "Unit" columns).
`har_mapping_info`	Data frame or NULL. Mapping information for HAR variables (with "Variable", "Description", and "Unit" columns).
`sl4_extract_method`	Character. SL4 extraction method. Options: "get_data_by_dims", "get_data_by_var", or "group_data_by_dims".
`har_extract_method`	Character. HAR extraction method. Options: "get_data_by_dims", "get_data_by_var", or "group_data_by_dims".
`sl4_priority`	Optional list. Priority rules for SL4 data grouping.
`har_priority`	Optional list. Priority rules for HAR data grouping.
`region_select`	Optional character vector. Specifies regions to filter the data.
`sector_select`	Optional character vector. Specifies sectors to filter the data.

`subtotal_level` Logical. If TRUE, includes subtotal data. Default is FALSE.

`plot_data` Logical. If TRUE, prepares data for plotting and assigns to variables.

`output_formats` Character vector or list. Exports data in these formats (valid: "csv", "stata", "rds", "txt").

`sl4_output_name`
Character. Variable name for SL4 plotting data if generating plot data. Default is "sl4.plot.data".

`har_output_name`
Character. Variable name for HAR plotting data if generating plot data. Default is "har.plot.data".

`macro_output_name`
Character. Variable name for GTAP macro data if generating plot data. Default is "GTAPMacro".

`add_scenario_ranking`
Logical. If TRUE, adds a numeric ranking column to all data frames. Default is FALSE.

`rank_column` Character. Name of the column to add with scenario ranking information. Default is "ScenarioRank".

Details

This function automates the workflow for processing GTAP model outputs, with flexible output options and optional filtering for region- and sector-specific data.

The key parameters ‘process_sl4_vars‘ and ‘process_har_vars‘ accept three different input types:

- **Data frame**: Contains variable mappings with required "Variable" column. When provided, only specified variables will be extracted.
- **NULL**: Extracts all available variables from the respective file type.
- **FALSE**: Completely skips processing of that file type, allowing the function to focus only on the other file type.

The ‘mapping_info‘ parameter controls how descriptions and units are assigned:

- **GTAPv7**: Uses standard GTAPv7 definitions (default).
- **Yes**: Uses only the supplied descriptions and units from ‘sl4_mapping_info‘ / ‘har_mapping_info‘.
- **No**: Does not add any descriptions or units.
- **Mix**: Prioritizes supplied descriptions and units, falling back to GTAPv7 for any missing values.

Value

A processed dataset, with options for exporting and visualization.

Author(s)

Pattawee Puangchit

See Also

`add_mapping_info`, `gtap_macros_data`

Examples

```
# Input Path:
input_path <- system.file("extdata/in", package = "GTAPViz")

# GTAP Macro Variables from 2 .sl4 Files named (EXP1, EXP2)
# Note: No need to add .sl4 to the experiment name
gtap_data <- auto_gtap_data(experiment = c("EXP1", "EXP2"),
                            input_path = input_path, subtotal_level = FALSE,
                            process_sl4_vars = NULL, process_har_vars = NULL,
                            mapping_info = "GTAPv7", plot_data = TRUE)
```

---

comparison_plot *Create Comparative Bar Charts from HAR and SL4 Data*

Description

Generates comparative bar charts using GTAP data, allowing multiple visualization options such as panel facets, split grouping, and customizable styles.

Usage

```
comparison_plot(
  data,
  filter_var = NULL,
  x_axis_from,
  split_by = NULL,
  panel_var = "Experiment",
  variable_col = "Variable",
  unit_col = "Unit",
  desc_col = "Description",
  invert_pane = FALSE,
  separate_figure = FALSE,
  var_name_by_description = FALSE,
  add_var_info = FALSE,
  output_path = NULL,
  export_picture = TRUE,
  export_as_pdf = FALSE,
  export_config = NULL,
  plot_style_config = NULL
)
```

Arguments

data	A data frame or a list of data frames containing GTAP results.
filter_var	Vector or data frame. If a vector, filters the values in ‘x_axis_from‘. If a data frame, filters ‘value_col‘ based on matching ‘variable_col‘ values.
x_axis_from	Character. Column name for x-axis categories (e.g., "REG", "Sector").
split_by	Character or vector. Column name(s) to generate separate plots for each unique value (e.g., "COMM", "REG", "Variable"). NULL creates a single aggregated plot, appropriate for macro-level analysis.

`panel_var` Character. Column for panel facets (default: "Experiment").

`variable_col` Character. Column containing variable identifiers (default: "Variable").

`unit_col` Character. Column containing unit information (default: "Unit").

`desc_col` Character. Column containing variable descriptions (default: "Description").

`invert_pane` Logical. If TRUE, creates horizontal bars instead of vertical ones (default: FALSE).

`separate_figure` Logical. If TRUE, generates separate figures per panel value (default: FALSE).

`var_name_by_description` Logical. If TRUE, uses descriptions instead of variable codes in titles (default: FALSE).

`add_var_info` Logical. If TRUE, adds the variable code in parentheses after the description (default: FALSE).

`output_path` Character. Directory path for saving output files. If NULL, plots are only returned in R.

`export_picture` Logical. If TRUE, exports plots as image files (default: TRUE).

`export_as_pdf` Logical or "merged". If TRUE, exports separate PDFs; if "merged", creates a multi-page PDF; if FALSE, skips PDF (default: FALSE).

`export_config` List. Export settings, including:

- 'width': Output file width (in inches).
- 'height': Output file height (in inches).
- Additional settings—see '?get_export_config'.

`plot_style_config` List. Custom plot styles—see '?get_plot_style_config'.

Value

A ggplot2 object for a single plot or a list of ggplot2 objects for multiple plots.

Author(s)

Pattawee Puangchit

See Also

`get_plot_style_config`, `get_export_config`, `detail_plot`, `stack_plot`

Examples

```
# Input Path:
input_path <- system.file("extdata/in", package = "GTAPViz")

# GTAP Macro Variables from 2 .sl4 Files named (EXP1, EXP2)
# Note: No need to add .sl4 to the experiment name
gtap_data <- auto_gtap_data(experiment = c("EXP1", "EXP2"),
                            input_path = input_path, subtotal_level = FALSE,
                            process_sl4_vars = NULL, process_har_vars = NULL,
                            mapping_info = "GTAPv7", plot_data = TRUE)

# Basic usage with data frame
```

```
p1 <- comparison_plot(
  data = sl4.plot.data[["1D"]][["Region"]],
  x_axis_from = "Region",
  panel_var = "Experiment",
  filter_var = c("qgdp", "EV"),
  output_path = tempdir(),
  export_picture = FALSE,
  export_as_pdf = FALSE,
)

# Split by commodity with custom styling and export options
p2 <- comparison_plot(
  data = sl4.plot.data[["1D"]][["Region"]],
  x_axis_from = "Region",
  split_by = "Variable",
  panel_var = "Experiment",
  filter_var = c("qgdp", "EV"),
  var_name_by_description = TRUE,
  output_path = tempdir(),
  export_picture = FALSE,
  export_as_pdf = FALSE,
  export_config = list(
    file_name = "commodity_impacts",
    width = 12,
    height = 8
  ),
  plot_style_config = list(
    color_tone = "economic",
    title_size = 16,
    show_grid_major_y = TRUE
  )
)
```

---

convert_units *Convert Units in GTAP Data*

Description

Converts values in a dataset to different units based on predefined transformations or custom scaling. Supports manual and automatic conversions for economic and trade-related metrics.

Usage

```
convert_units(
  data,
  change_unit_from = NULL,
  change_unit_to = NULL,
  adjustment = NULL,
  value_col = "Value",
  unit_col = "Unit",
  variable_select = NULL,
  variable_col = "Variable",
  scale_auto = NULL
)
```

Arguments

data	A data structure (list, data frame, or nested combination).
change_unit_from	Character vector. Units to be converted (case-insensitive).
change_unit_to	Character vector. Target units corresponding to ‘change_unit_from‘.
adjustment	Character or numeric vector. Specifies conversion operations (e.g., ‘"/1000"‘ to convert million to billion).
value_col	Character. Column name containing values to adjust (default: ‘"Value"‘).
unit_col	Character. Column name containing unit information (default: ‘"Unit"‘).
variable_select	Optional character vector. If provided, only these variables are converted.
variable_col	Character. Column name containing variable identifiers (default: ‘"Variable"‘).
scale_auto	Optional character vector of predefined conversion rules: - ‘"mil2bil"‘: Converts million USD to billion USD (divides by 1000). - ‘"bil2mil"‘: Converts billion USD to million USD (multiplies by 1000). - ‘"pct2frac"‘: Converts percent to fraction (divides by 100). - ‘"frac2pct"‘: Converts fraction to percent (multiplies by 100).

Details

If both ‘change_unit_from‘ and ‘scale_auto‘ are provided, the function prompts the user to choose between manual and automatic conversion.

Value

A data structure with values converted to the specified units.

Author(s)

Pattawee Puangchit

See Also

add_mapping_info, rename_value, sort_plot_data

Examples

```
# Load Sample Data:
sl4_data1 <- HARplus::load_sl4x(system.file("extdata/in", "EXP1.sl4",
                                package = "GTAPViz"))

# Get Data by Variable Name
sl4_data1 <- HARplus::get_data_by_var(c("qgdp", "EV"),sl4_data1)

# Convert million USD to billion USD
gtap_data <- convert_units(sl4_data1,
  change_unit_from = "million USD",
  change_unit_to = "billion USD",
  adjustment = "/1000"
)

# Automatic conversion from percent to fraction
```

```
gtap_data <- convert_units(sl4_data1, scale_auto = "pct2frac")
```

`detail_plot` *Create Comprehensive Bar Charts from HAR and SL4 Data*

Description

Generates detailed bar charts to visualize the distribution of impacts across multiple dimensions. The function supports top impact filtering, color coding, and flexible visualization settings.

Usage

```
detail_plot(
  data,
  filter_var = NULL,
  x_axis_from,
  split_by = NULL,
  panel_var = "Experiment",
  variable_col = "Variable",
  unit_col = "Unit",
  desc_col = "Description",
  var_name_by_description = FALSE,
  add_var_info = FALSE,
  top_impact = NULL,
  invert_pane = FALSE,
  separate_figure = FALSE,
  output_path = NULL,
  export_picture = TRUE,
  export_as_pdf = FALSE,
  export_config = NULL,
  plot_style_config = NULL
)
```

Arguments

`data`	A data frame or a list of data frames containing GTAP results.
`filter_var`	Vector or data frame. If a vector, filters the values in ‘x_axis_from‘. If a data frame, filters ‘value_col‘ based on matching ‘variable_col‘ values.
`x_axis_from`	Character. Column name for x-axis categories (e.g., "REG", "Sector").
`split_by`	Character or vector. Column name(s) to generate separate plots for each unique value (e.g., "COMM", "REG", "Variable"). NULL creates a single aggregated plot, appropriate for macro-level analysis.
`panel_var`	Character. Column for panel facets (default: "Experiment").
`variable_col`	Character. Column containing variable identifiers (default: "Variable").
`unit_col`	Character. Column containing unit information (default: "Unit").
`desc_col`	Character. Column containing variable descriptions (default: "Description").
`var_name_by_description`	Logical. If TRUE, uses descriptions instead of variable codes in titles (default: FALSE).

`add_var_info` Logical. If TRUE, adds the variable code in parentheses after the description (default: FALSE).

`top_impact` Numeric or NULL. If specified, shows only the top N impactful values; NULL shows all values.

`invert_pane` Logical. If TRUE, creates horizontal bars instead of vertical ones (default: FALSE).

`separate_figure` Logical. If TRUE, generates separate figures per panel value (default: FALSE).

`output_path` Character. Directory path for saving output files. If NULL, plots are only returned in R.

`export_picture` Logical. If TRUE, exports plots as image files (default: TRUE).

`export_as_pdf` Logical or "merged". If TRUE, exports separate PDFs; if "merged", creates a multi-page PDF; if FALSE, skips PDF (default: FALSE).

`export_config` List. Export settings, including:

- 'width': Output file width (in inches).
- 'height': Output file height (in inches).
- Additional settings—see '?get_export_config'.

`plot_style_config` List. Custom plot styles—see '?get_plot_style_config'.

Value

A ggplot2 object for a single plot or a list of ggplot2 objects for multiple plots.

Author(s)

Pattawee Puangchit

See Also

get_plot_style_config, get_export_config, comparison_plot, stack_plot

Examples

```
# Input Path:
input_path <- system.file("extdata/in", package = "GTAPViz")

# GTAP Macro Variables from 2 .sl4 Files named (EXP1, EXP2)
# Note: No need to add .sl4 to the experiment name
gtap_data <- auto_gtap_data(experiment = c("EXP1", "EXP2"),
                            input_path = input_path, subtotal_level = FALSE,
                            process_sl4_vars = NULL, process_har_vars = NULL,
                            mapping_info = "GTAPv7", plot_data = TRUE)
# Basic usage with data frame
detail_plot(sl4.plot.data[["2D"]],
            x_axis_from = "Sector",
            split_by = "Region",
            filter_var = "qo",

            top_impact = NULL,
            var_name_by_description = TRUE,
```

```
  invert_pane = TRUE,
  separate_figure = FALSE,

  export_config = list(
    width = 45,
    height = 20
  ),

  export_picture = FALSE,
  export_as_pdf = FALSE,
  output_path = tempdir(),

  plot_style_config = list(
    positive_color = "#2E8B57",
    negative_color = "#CD5C5C",
    panel_rows = 1,
    panel_cols = NULL,
    show_axis_titles_on_all_facets = FALSE,
    y_axis_text_size = 25,
    bar_width = 0.6,
    all_font_size = 1.1
  ))
```

`get_all_config` *Get Plot and Export Configurations*

Description

Returns a list containing plot style configuration and export configuration. The function supports retrieving a single plot style or all available styles.

Usage

```
get_all_config(
  plot_style = "default",
  config = NULL,
  export_config = NULL,
  printing = FALSE
)
```

Arguments

`plot_style`	Character. The plot style to retrieve: "comparison", "detail", "stack", or "all". Default is "all", which returns configurations for all styles.
`config`	List. Optional custom style configuration to override defaults.
`export_config`	List. Optional custom export configuration to override defaults.
`printing`	Logical. If TRUE, prints a formatted code snippet that can be copied and pasted to recreate the configuration. Default is FALSE.

Value

A list with two components:

- `plot_style_config`: Plot style configuration(s). If `plot_style="all"`, contains a nested list with configurations for all styles.
- `export_config`: Export configuration as a data frame.

Author(s)

Pattawee Puangchit

See Also

`get_plot_style_config`, `get_export_config`, `comparison_plot`, `detail_plot`, `stack_plot`

Examples

```
# Get all plot configurations with default settings
all_configs <- get_all_config()

# Get only comparison plot configuration
comp_config <- get_all_config("default")
```

---

get_color_palette *Print and Visualize Themed Color Palettes*

Description

This function prints and visualizes predefined color palettes. Users can specify a ‘color_tone‘ and ‘palette_type‘ to display the corresponding colors. Additionally, calling ‘color_tone = "all"‘ returns a list of callable functions, where each entry dynamically generates a color palette visualization.

Usage

```
get_color_palette(color_tone = NULL, palette_type = "qualitative")
```

Arguments

`color_tone` Character. The name of the predefined color theme. Default is ‘NULL‘, which requires specification. Available themes include:

- **Academic**: A balanced set of colors for research-oriented visuals.
- **Purdue**: Themed after Purdue University branding.
- **Colorblind**: Designed for colorblind-friendly visualization.
- **Economic**: Used for economic data visualization.
- **Trade**: Suited for trade-related data.
- **GTAP**: Based on Global Trade Analysis Project visuals.
- **GTAP2**: An alternative GTAP color set.
- **Earth**: Inspired by natural, earthy tones.
- **Vibrant**: High-contrast and energetic colors.
- **Bright**: Bright and playful color combinations.

- **Minimal**: Monochrome and muted colors for minimalistic designs.
- **Energetic**: Warm and dynamic tones.
- **Pastel**: Soft pastel shades.
- **Spring**: Fresh, floral-inspired hues.
- **Summer**: Sunny, warm color gradients.
- **Winter**: Cool, icy tones for winter visuals.
- **Fall**: Autumn-inspired warm color palette.
- **Blue_mono**, **Green_mono**, **Red_mono**, **Grey_mono**: Monochromatic shades for respective colors.

Use ‘"all"‘ to return a list of callable functions for all palettes.

`palette_type` Character. The type of color palette to use. Options include:

- **"qualitative"** - Best for categorical data.
- **"sequential"** - Used for ordered, continuous scales.
- **"diverging"** - Ideal for highlighting contrasts.

Default is ‘"qualitative"‘.

Details

The function retrieves colors from ‘.create_color_palette()‘ and provides a visual preview of the selected theme.

If ‘color_tone = "all"‘, the function returns a **list of functions**, where calling any element (e.g., ‘all_palettes$winter()‘) generates the corresponding color palette visualization.

If the requested ‘color_tone‘ does not exist or is empty, the function throws an error.

Value

- If a specific ‘color_tone‘ is provided, it prints the color palette and returns ‘NULL‘.
- If ‘color_tone = "all"‘, it returns a **list of callable functions** to visualize each palette on demand.

Examples

```
## Not run:
# Print & visualize a specific palette
get_color_palette("winter")

# Print & visualize another palette with different types
get_color_palette("fall", "sequential")
get_color_palette("academic", "diverging")

# Get all palettes as a list of callable functions
all_palettes <- get_color_palette("all")

# Click or call a specific palette function to view its colors
all_palettes$winter()   # View the winter palette
all_palettes$fall()     # View the fall palette
all_palettes$gtap()     # View the GTAP palette

## End(Not run)
```

`get_export_config` *Get Export Configuration Options*

Description

Returns documentation and default values for export configuration options used in plotting functions.

Usage

```
get_export_config(as_dataframe = TRUE, printing = FALSE)
```

Arguments

`as_dataframe`	Logical. Whether to return settings as a dataframe. Default is FALSE.
`printing`	Logical. If TRUE, prints a formatted code snippet that can be copied and pasted to recreate the configuration. Default is FALSE.

Details

Export Configuration Parameters - ‘file_name‘: Character. Base name for exported files. Default: ‘"gtap_plots"‘ - ‘width‘: Numeric or ‘NULL‘. Plot width in inches. Default: ‘NULL‘ (automatically calculated) - ‘height‘: Numeric or ‘NULL‘. Plot height in inches. Default: ‘NULL‘ (automatically calculated) - ‘dpi‘: Numeric. Resolution for PNG export. Default: ‘300‘ - ‘bg‘: Character. Background color. Default: ‘"white"‘ - ‘limitsize‘: Logical. Whether to limit size. Default: ‘FALSE‘

Value

If `as_dataframe` is TRUE, returns a dataframe with export configuration settings. Otherwise, returns a list with configuration settings.

Author(s)

Pattawee Puangchit

See Also

`comparison_plot`, `detail_plot`, `stack_plot`

Examples

```
# Get export configuration as list
export_info <- get_export_config()

# Get as a formatted dataframe
export_df <- get_export_config(as_dataframe = TRUE)
```

`get_plot_style_config` *Get Plot Style Configuration*

Description

Returns configuration settings for plot styles, with options to view as a structured dataframe or to look up specific parameters. Also provides parameter validation for custom configurations.

Usage

```
get_plot_style_config(
  plot_type = "default",
  parameter_name = NULL,
  show_docs = FALSE,
  validate_custom = NULL,
  as_dataframe = FALSE,
  printing = TRUE
)
```

Arguments

`plot_type`	Character. Type of plot: "default" (default).
`parameter_name`	Character or NULL. Name of specific parameter to return information about.
`show_docs`	Logical. Whether to include documentation in the output.
`validate_custom`	List or NULL. Custom configuration settings to validate.
`as_dataframe`	Logical. Whether to return settings as a dataframe.
`printing`	Logical. If TRUE, prints a formatted code snippet that can be copied and pasted to recreate the configuration. Default is FALSE.

Details

This function applies default styling for different types of plots and allows users to customize the appearance. The parameters are grouped as follows:

Title Settings - ‘show_title‘: Logical. Show or hide the plot title. Default: ‘TRUE‘ - ‘title_face‘: Character. Font face (‘"bold"‘, ‘"plain"‘, ‘"italic"‘). Default: ‘"bold"‘ - ‘title_size‘: Numeric. Font size of title. Default: ‘20‘ - ‘title_hjust‘: Numeric. Horizontal alignment (0 = left, 1 = right). Default: ‘0.5‘ - ‘add_unit_to_title‘: Logical. Append unit to title if applicable. Default: ‘TRUE‘ - ‘title_margin‘: Numeric vector ‘c(top, right, bottom, left)‘. Default: ‘c(10, 0, 10, 0)‘ - ‘title_format‘: List or NULL. Formatting options for the title, with elements:

- `type`: Character. One of "prefix", "suffix", "full", or "dynamic".
- `text`: Character. Text to add, or column names used for dynamic titles.
- `sep`: Character. Separator used for dynamic titles. Default: " - ".

X-Axis Settings - ‘show_x_axis_title‘: Logical. Show or hide x-axis title. Default: ‘TRUE‘ - ‘x_axis_title_face‘: Character. Font face for x-axis title. Default: ‘"bold"‘ - ‘x_axis_title_size‘: Numeric. Font size of x-axis title. Default: ‘16‘ - ‘x_axis_title_margin‘: Numeric vector ‘c(top, right, bottom, left)‘. Default: ‘c(25, 25, 0, 0)‘ - ‘show_x_axis_labels‘: Logical. Show or hide x-axis labels. Default: ‘TRUE‘ - ‘x_axis_text_face‘: Character. Font face for x-axis labels. Default:

‘"bold"‘ - ‘x_axis_text_size‘: Numeric. Font size of x-axis labels. Default: ‘14‘ - ‘x_axis_text_angle‘: Numeric. Angle of x-axis labels. Default: ‘45‘ - ‘x_axis_text_hjust‘: Numeric. Horizontal justification of x-axis labels. Default: ‘1‘ - ‘x_axis_description‘: Character. Optional description for the x-axis. Default: ‘""‘

Y-Axis Settings - ‘show_y_axis_title‘: Logical. Show or hide y-axis title. Default: ‘TRUE‘ - ‘y_axis_title_face‘: Character. Font face for y-axis title. Default: ‘"bold"‘ - ‘y_axis_title_size‘: Numeric. Font size of y-axis title. Default: ‘16‘ - ‘y_axis_title_margin‘: Numeric vector ‘c(top, right, bottom, left)‘. Default: ‘c(25, 25, 0, 0)‘ - ‘show_y_axis_labels‘: Logical. Show or hide y-axis labels. Default: ‘TRUE‘ - ‘y_axis_text_face‘: Character. Font face for y-axis labels. Default: ‘"plain"‘ - ‘y_axis_text_size‘: Numeric. Font size of y-axis labels. Default: ‘14‘ - ‘y_axis_text_angle‘: Numeric. Angle of y-axis labels. Default: ‘0‘ - ‘y_axis_text_hjust‘: Numeric. Horizontal justification of y-axis labels. Default: ‘0‘ - ‘y_axis_description‘: Character. Optional description for the y-axis. Default: ‘""‘ - ‘show_axis_titles_on_all_facets‘: Logical. Show axis titles on all facets. Default: ‘TRUE‘

Value Label Settings - ‘show_value_labels‘: Logical. Show or hide value labels. Default: ‘TRUE‘ - ‘value_label_face‘: Character. Font face for value labels. Default: ‘"plain"‘ - ‘value_label_size‘: Numeric. Font size of value labels. Default: ‘5‘ - ‘value_label_position‘: Character. Position of value labels (‘"above"‘, ‘"outside"‘, ‘"top"‘). Default: ‘"above"‘ - ‘value_label_decimal_places‘: Numeric. Number of decimal places in value labels. Default: ‘2‘

Legend Settings - ‘show_legend‘: Logical. Show or hide legend. Default: ‘FALSE‘ - ‘show_legend_title‘: Logical. Show or hide legend title. Default: ‘FALSE‘ - ‘legend_position‘: Character. Legend position (‘"none"‘, ‘"bottom"‘, ‘"right"‘). Default: ‘"none"‘ - ‘legend_title_face‘: Character. Font face for legend title. Default: ‘"bold"‘ - ‘legend_text_face‘: Character. Font face for legend text. Default: ‘"plain"‘ - ‘legend_text_size‘: Numeric. Font size for legend text. Default: ‘14‘

Panel Strip Settings - ‘strip_face‘: Character. Font face for panel strip. Default: ‘"bold"‘ - ‘strip_text_size‘: Numeric. Font size for panel strip. Default: ‘16‘ - ‘strip_background‘: Character. Background color of strip. Default: ‘"lightgrey"‘ - ‘strip_text_margin‘: Numeric vector ‘c(top, right, bottom, left)‘. Default: ‘c(10, 0, 10, 0)‘

Panel Layout - ‘panel_spacing‘: Numeric. Spacing between panels. Default: ‘2‘ - ‘panel_rows‘: Numeric or ‘NULL‘. Number of rows in panel layout. Default: ‘NULL‘ - ‘panel_cols‘: Numeric or ‘NULL‘. Number of columns in panel layout. Default: ‘NULL‘ - ‘theme‘: ggplot2 theme object or ‘NULL‘. Custom ggplot theme. Default: ‘NULL‘

Color and Grid Settings - ‘color_tone‘: Character or ‘NULL‘. Base color theme. Default: ‘NULL‘ - ‘positive_color‘: Character. Color for positive values. Default: ‘"#2E8B57"‘ - ‘negative_color‘: Character. Color for negative values. Default: ‘"#CD5C5C"‘ - ‘background_color‘: Character. Background color of plot. Default: ‘"white"‘ - ‘grid_color‘: Character. Color of grid lines. Default: ‘"grey90"‘ - ‘show_grid_major_x‘: Logical. Show major grid lines on x-axis. Default: ‘FALSE‘ - ‘show_grid_major_y‘: Logical. Show major grid lines on y-axis. Default: ‘TRUE‘ - ‘show_grid_minor_x‘: Logical. Show minor grid lines on x-axis. Default: ‘FALSE‘ - ‘show_grid_minor_y‘: Logical. Show minor grid lines on y-axis. Default: ‘FALSE‘

Zero Line Settings - ‘show_zero_line‘: Logical. Show or hide zero line. Default: ‘TRUE‘ - ‘zero_line_type‘: Character. Line type (‘"solid"‘, ‘"dashed"‘, ‘"dotted"‘). Default: ‘"dashed"‘ - ‘zero_line_color‘: Character. Color of zero line. Default: ‘"black"‘ - ‘zero_line_size‘: Numeric. Line thickness of zero line. Default: ‘0.5‘ - ‘zero_line_position‘: Numeric. Position of the zero line. Default: ‘0‘

Bar Chart Settings - ‘bar_width‘: Numeric. Width of bars. Default: ‘0.9‘ - ‘bar_spacing‘: Numeric. Spacing between bars. Default: ‘0.9‘

Scale Settings - ‘scale_limit‘: Numeric vector of length 2. Manual limits for value axis. Example: ‘c(-10, 10)‘ - ‘scale_increment‘: Numeric. Step size for axis tick marks. Example: ‘2‘

Scale Expansion Settings - 'expansion_y_mult': Numeric vector. Y-axis expansion. Default: 'c(0.05, 0.1)' - 'expansion_x_mult': Numeric vector. X-axis expansion. Default: 'c(0.05, 0.05)'

All Font Adjustment - 'all_font_size': Numeric. Master control for all font sizes. Default: '1'

Plot Margin Settings - 'plot.margin': Numeric vector 'c(top, right, bottom, left)'. Margins around the entire plot. Default: 'c(10, 25, 10, 10)'

Value

If `parameter_name` is specified, returns a list with the value and documentation for that parameter. If `validate_custom` is specified, returns the validated configuration list. If `as_dataframe` is TRUE, returns a dataframe with configuration settings. Otherwise, returns a list with all configuration settings.

Author(s)

Pattawee Puangchit

See Also

`comparison_plot`, `detail_plot`, `stack_plot`

Examples

```
# Get all default configuration
get_plot_style_config(printing = TRUE)

# Get information about a specific parameter
param_info <- get_plot_style_config("default", "bar_width")

# Get as structured dataframe
config_df <- get_plot_style_config("default", as_dataframe = TRUE)

# Validate custom configuration
custom_config <- list(title_size = 24, bar_width = 0.7, plot.margin = c(10, 50, 10, 10))
validated <- get_plot_style_config("default", validate_custom = custom_config)
```

---

gtap_macros_data *Extract and Aggregate Scalar Macroeconomic Variables*

Description

Extracts scalar macroeconomic variables from multiple SL4 datasets and aggregates them into a structured data frame.

Usage

```
gtap_macros_data(
  select_var = NULL,
  experiment = NULL,
  input_path = NULL,
  output_path = NULL,
  output_formats = NULL,
  subtotal_level = FALSE
)
```

Arguments

select_var	Character vector (optional). List of specific variable names to filter from the final result. If NULL, all variables are returned.
experiment	Character vector. List of experiment names corresponding to SL4 files.
input_path	Character. Path to the directory containing SL4 files.
output_path	Character (optional). Directory to save exported data.
output_formats	Character vector (optional). List of output formats (e.g., "csv", "xlsx").
subtotal_level	Logical. Whether to include subtotal levels in the processed data.

Value

A sorted data frame containing processed GTAP macro data.

Author(s)

Pattawee Puangchit

See Also

add_mapping_info, auto_gtap_data

Examples

```
# Input Path:
input_path <- system.file("extdata/in", package = "GTAPViz")

# GTAP Macro Variables from 2 .sl4 Files named (EXP1, EXP2)
# Note: No need to add .sl4 to the experiment name
gtap_macro <- gtap_macros_data(NULL, experiment = c("EXP1", "EXP2"),
                               input_path = input_path, subtotal_level = FALSE)
```

`pivot_table_with_filter` *Export Data as an Excel Pivot Table*

Description

Exports a dataset to an Excel file with both raw data and a generated pivot table.

Usage

```
pivot_table_with_filter(
  data,
  filter = NULL,
  rows = NULL,
  cols = NULL,
  data_fields = "Value",
  raw_sheet_name = "RawData",
  pivot_sheet_name = "PivotTable",
  dims = "A5",
  export = TRUE,
  output_path = getwd(),
  workbook_name = "GTAP_PivotTable.xlsx"
)
```

Arguments

`data`	Data frame. The dataset to be exported.
`filter`	Character vector (optional). Columns to be used as filter fields in the pivot table.
`rows`	Character vector (optional). Columns to be used as row fields in the pivot table.
`cols`	Character vector (optional). Columns to be used as column fields in the pivot table.
`data_fields`	Character. The data field(s) to be summarized in the pivot table (default: ‘"Value"‘).
`raw_sheet_name`	Character. Name of the sheet containing raw data (default: ‘"RawData"‘).
`pivot_sheet_name`	Character. Name of the sheet containing the pivot table (default: ‘"PivotTable"‘).
`dims`	Character. Cell reference where the pivot table starts (default: ‘"A3"‘).
`export`	Logical. Whether to save the Excel file (default: ‘TRUE‘).
`output_path`	Character. Directory where the file should be saved (default: current working directory).
`workbook_name`	Character. Name of the output Excel file (default: ‘"GTAP_PivotTable.xlsx"‘).

Details

This function creates an Excel workbook with: - A raw data sheet (‘raw_sheet_name‘) containing the provided dataset. - A pivot table sheet (‘pivot_sheet_name‘) generated based on specified row, column, and data fields.

If ‘export = TRUE‘, the function saves the workbook to the specified ‘output_path‘.

Value

An excel workbook object containing both raw data and the pivot table.

Author(s)

Pattawee Puangchit

Examples

```
# Input Path
input_path <- system.file("extdata/in", package = "GTAPViz")

# Prepares data from SL4 files for EXP1 and EXP2
auto_gtap_data(
  experiment = c("EXP1", "EXP2"),
  input_path = input_path,
  subtotal_level = FALSE,
  process_sl4_vars = c("qo", "qgdp"),
  process_har_vars = FALSE,
  mapping_info = "GTAPv7",
  plot_data = TRUE
)

# Generate comparison table
pivot_table_with_filter(
  data = sl4.plot.data[["2D"]][["Sector"]],
  filter = c("Variable", "Unit"),
  rows = c("Region", "Sector"),
  cols = c("Experiment"),
  data_fields = "Value",
  raw_sheet_name = "Raw_Data",
  pivot_sheet_name = "Sector_Pivot",
  export = TRUE,
  output_path = tempdir(),
  workbook_name = "Sectoral_Impact_Analysis.xlsx"
)
```

---

rename_GTAP_bilateral *Rename GTAP Bilateral Trade Columns*

Description

Renames bilateral trade columns in GTAP data to standardized names, ensuring consistency in regional trade flows.

Usage

```
rename_GTAP_bilateral(data)
```

Arguments

data	Data structure containing GTAP bilateral trade data.

Value

The same data structure with renamed bilateral trade columns.

Author(s)

Pattawee Puangchit

See Also

add_mapping_info, convert_units, sort_plot_data

Examples

```
# Load Sample Data:
sl4_data1 <- HARplus::load_sl4x(system.file("extdata/in", "EXP1.sl4",
                                package = "GTAPViz"))
# Get Data by Variable Name
sl4_data1 <- HARplus::get_data_by_var("qxs",sl4_data1)

# Rename bilateral trade columns in a GTAP dataset
gtap_data <- rename_GTAP_bilateral(sl4_data1)
```

---

rename_value *Rename Values in a Column*

Description

Replaces specific values in a column based on a provided mapping file. Supports renaming across nested data structures and preserves factor levels.

Usage

```
rename_value(data, column_name = NULL, mapping.file)
```

Arguments

data	Data structure (data frame, list, or nested combination).
column_name	Character. Column to modify. If 'NULL', the function extracts it from 'mapping.file'.
mapping.file	Data frame with '"OldName"' and '"NewName"' columns for renaming.

Value

The same data structure with specified values replaced.

Author(s)

Pattawee Puangchit

See Also

add_mapping_info, convert_units, sort_plot_data

Examples

```
# Load Sample Data:
sl4_data1 <- HARplus::load_sl4x(system.file("extdata/in", "EXP1.sl4",
                                package = "GTAPViz"))
# Get Data by Variable Name
sl4_data1 <- HARplus::get_data_by_var(c("qgdp", "EV"),sl4_data1)

# Rename variables in a dataset
rename_map <- data.frame(OldName = c("qgdp", "EV"),
                         NewName = c("Real GDP", "Welfare"))
gtap_data <- rename_value(sl4_data1, column_name = "Variable",
                          mapping.file = rename_map)
```

---

report_table *Generate a Structured Report Table*

Description

Transforms multiple datasets into wide-format tables based on defined pivot columns, hierarchical grouping, and renaming rules. Supports optional subtotal filtering and exporting to Excel.

Usage

```
report_table(
  data_list,
  pivot_col,
  total_column = FALSE,
  export_table = FALSE,
  separate_file = FALSE,
  output_path = NULL,
  sheet_names = NULL,
  include_units = FALSE,
  component_exclude = NULL,
  group_by = NULL,
  rename_cols = NULL,
  var_name_by_description = TRUE,
  add_var_info = FALSE,
  decimal = 2,
  unit_select = NULL,
  separate_sheet_by = NULL,
  subtotal_level = FALSE,
  repeat_label = FALSE,
  workbook_name = "detail_results",
  add_group_line = FALSE
)
```

Arguments

data_list	A named list of data frames to process.

pivot_col	A named list specifying the column to pivot into a wide format for each dataset. Each dataset can have only one pivot column. Example: `pivot_col = list(A = "COLUMN", E1 = "PRICES")`
total_column	Logical. If 'TRUE', adds a "Total" column summing numeric values.
export_table	Logical. If 'TRUE', saves the output as an Excel file.
separate_file	Logical. If 'TRUE', saves each dataset as a separate Excel file.
output_path	Character. Directory for saving Excel files when 'export_table = TRUE'.
sheet_names	Optional named list for custom sheet names.
include_units	Logical. If 'TRUE', includes "Unit" as a grouping column if applicable.
component_exclude	Optional character vector specifying pivoted values to exclude.
group_by	A named list defining hierarchical grouping for each dataset. The order of columns in each list determines the priority. Example: `group_by = list(A = list("Experiment", "REG"), E1 = list("Experiment", "REG", "COMM"))`
rename_cols	A named list for renaming columns across **all** datasets. Example: `rename_cols = list("REG" = "Region", "COMM" = "Commodities", "Experiment" = "Scenario")`
var_name_by_description	Logical. If 'TRUE', replaces variable codes with descriptions when available.
add_var_info	Logical. If 'TRUE', appends variable codes in parentheses after descriptions.
decimal	Numeric. Number of decimal places for rounding values.
unit_select	Optional character. Specifies a unit to filter the dataset.
separate_sheet_by	Optional column name to split sheets in Excel. If defined, each unique value in the specified column gets its own sheet. Example: `separate_sheet_by = "Scenario"`.
subtotal_level	Logical. If 'TRUE', includes all subtotal values; otherwise, keeps only 'TOTAL' rows.
repeat_label	Logical. If 'TRUE', repeats the first group column in exports for clarity.
workbook_name	Character. Name of the Excel workbook (without extension).
add_group_line	Logical. If 'TRUE', adds a thin line after each group in the exported table.

Value

A named list of transformed data frames. If 'export_table = TRUE', tables are saved as Excel files.

Author(s)

Pattawee Puangchit

See Also

add_mapping_info, convert_units, rename_value

Examples

```
# Input Path
input_path <- system.file("extdata/in", package = "GTAPViz")

# Prepares data from SL4 files for EXP1 and EXP2
auto_gtap_data(
  experiment = c("EXP1", "EXP2"),
  input_path = input_path,
  subtotal_level = FALSE,
  process_sl4_vars = c("qgdp", "EV", "qo"),
  process_har_vars = FALSE,
  mapping_info = "GTAPv7",
  plot_data = TRUE
)

# Generate comparison table
report_table(
  data_list = sl4.plot.data[["1D"]],
  pivot_col = list(Region = "Variable"),
  group_by = list(Region = list("Experiment", "Region")),
  rename_cols = list("Experiment" = "Scenario"),

  total_column = FALSE,
  decimal = 4,
  subtotal_level = FALSE,
  repeat_label = FALSE,
  include_units = TRUE,

  var_name_by_description = TRUE,
  add_var_info = TRUE,
  add_group_line = FALSE,

  separate_sheet_by = "Unit",
  export_table = FALSE,
  output_path = tempdir(),
  separate_file = FALSE,
  workbook_name = "Comparison Table"
)
```

`sort_plot_data` *Sort GTAP Plot Data*

Description

Sorts data for plotting using flexible options for column ordering and value-based sorting. Works with data frames, lists of data frames, or nested data structures.

Usage

```
sort_plot_data(
  data,
  sort_columns = NULL,
  sort_by_value_desc = NULL,
```

```
  convert_to_factor = TRUE
)
```

Arguments

data	A data frame or list structure containing data to be sorted.
sort_columns	Named list. Specifies columns to sort by and their ordering. Each element should be a character vector of values in desired order. For example, 'list(Region = c("USA", "EU", "CHN")', 'Experiment = c("Base", "Shock1", "Shock2"))'.
sort_by_value_desc	Logical or NULL. Controls sorting by the "Value" column: - NULL (default): Don't sort by value, only use column-based sorting. - TRUE: After column-based sorting, sort by value in descending order. - FALSE: After column-based sorting, sort by value in ascending order.
convert_to_factor	Logical. Whether to convert sorted columns to factors with custom ordering. Default is TRUE, which preserves ordering in GTAP plotting functions.

Value

A data structure with the same form as the input, with all contained data frames sorted.

Author(s)

Pattawee Puangchit

See Also

add_mapping_info, convert_units, rename_GTAP_bilateral

Examples

```
# Load Sample Data:
sl4_data1 <- HARplus::load_sl4x(system.file("extdata/in", "EXP1.sl4",
                                package = "GTAPViz"))
# Get Data by Variable Name
sl4_data1 <- HARplus::get_data_by_var(c("qgdp","EV"),sl4_data1)

# Creating Sorting Rule
sorting_specs <- data.frame(REG = c("EastAsia", "SEAsia", "Oceania"))

# Sorting
sort_data <- sort_plot_data(sl4_data1, sort_columns = sorting_specs,
                            sort_by_value_desc = FALSE)
```

stack_plot *Create Stacked Bar Charts for Decomposition Analysis*

Description

Generates stacked bar charts to visualize value compositions across multiple dimensions. The function supports stacked and unstacked presentations for decomposition analysis.

Usage

```
stack_plot(
  data,
  filter_var = NULL,
  x_axis_from,
  stack_value_from,
  split_by = NULL,
  panel_var = "Experiment",
  variable_col = "Variable",
  unit_col = "Unit",
  desc_col = "Description",
  var_name_by_description = FALSE,
  add_var_info = FALSE,
  show_total = TRUE,
  unstack_plot = FALSE,
  top_impact = NULL,
  invert_pane = FALSE,
  separate_figure = FALSE,
  output_path = NULL,
  export_picture = TRUE,
  export_as_pdf = FALSE,
  export_config = NULL,
  plot_style_config = NULL
)
```

Arguments

data	A data frame or a list of data frames containing GTAP results.
filter_var	Vector or data frame. If a vector, filters the values in ‘x_axis_from‘. If a data frame, filters ‘value_col‘ based on matching ‘variable_col‘ values.
x_axis_from	Character. Column name for x-axis categories (e.g., "REG", "Sector").
stack_value_from	Character. Column containing stack component categories (e.g., "COMM" for commodities).
split_by	Character or vector. Column name(s) to generate separate plots for each unique value (e.g., "COMM", "REG", "Variable"). NULL creates a single aggregated plot, appropriate for macro-level analysis.
panel_var	Character. Column for panel facets (default: "Experiment").
variable_col	Character. Column containing variable identifiers (default: "Variable").
unit_col	Character. Column containing unit information (default: "Unit").

`desc_col` Character. Column containing variable descriptions (default: "Description").

`var_name_by_description`
Logical. If TRUE, uses descriptions instead of variable codes in titles (default: FALSE).

`add_var_info` Logical. If TRUE, adds the variable code in parentheses after the description (default: FALSE).

`show_total` Logical. If TRUE, displays total values above stacked bars (default: TRUE).

`unstack_plot` Logical. If TRUE, creates separate bar plots for each ‘x_axis_from‘ value instead of stacked bars (default: FALSE).

`top_impact` Numeric or NULL. If specified, shows only the top N impactful values; NULL shows all values.

`invert_pane` Logical. If TRUE, creates horizontal bars instead of vertical ones (default: FALSE).

`separate_figure`
Logical. If TRUE, generates separate figures per panel value (default: FALSE).

`output_path` Character. Directory path for saving output files. If NULL, plots are only returned in R.

`export_picture` Logical. If TRUE, exports plots as image files (default: TRUE).

`export_as_pdf` Logical or "merged". If TRUE, exports separate PDFs; if "merged", creates a multi-page PDF; if FALSE, skips PDF (default: FALSE).

`export_config` List. Export settings, including:

- ‘width‘: Output file width (in inches).
- ‘height‘: Output file height (in inches).
- Additional settings—see ‘?get_export_config‘.

`plot_style_config`
List. Custom plot styles—see ‘?get_plot_style_config‘.

Value

A ggplot2 object for a single plot or a list of ggplot2 objects for multiple plots.

Author(s)

Pattawee Puangchit

See Also

`get_plot_style_config`, `get_export_config`, `comparison_plot`, `detail_plot`

Examples

```
# Input Path:
input_path <- system.file("extdata/in", package = "GTAPViz")

# GTAP Macro Variables from 2 .sl4 Files named (EXP1, EXP2)
# Note: No need to add .sl4 to the experiment name
gtap_data <- auto_gtap_data(experiment = c("EXP1", "EXP2"),
                            input_path = input_path, subtotal_level = FALSE,
                            process_sl4_vars = NULL, process_har_vars = NULL,
                            mapping_info = "GTAPv7", plot_data = TRUE)
```

```
stack_plot(data = har.plot.data[["A"]],
           x_axis_from = "REG",
           stack_value_from = "COLUMN",
           split_by = FALSE,

           show_total = TRUE,
           unstack_plot = FALSE,

           var_name_by_description = TRUE,

           invert_pane = FALSE,
           separate_figure = FALSE,

           export_picture = FALSE,
           export_as_pdf = FALSE,
           export_config = list(
             width = 28,
             height = 15
           ),
           output_path = tempdir(),

           plot_style_config = list(
             color_tone = "gtap",
             panel_rows = 2,
             panel_cols = NULL,
             show_legend = TRUE,
             show_axis_titles_on_all_facets = FALSE
           ))
```

Index